# 03. Longest Substring Without Repeating Charact

: Sliding Window | LeetCode: https://leetcode.com/problems/longest-substring-w

# Problem

- Return the length of the longest substring of s with all distinct characters.
- Example: "abcabcbb" → 3 ("abc"); "pwwkew" → 3 ("wke").

# Intuition

- Keep a window [l, r] with no repeats. When s[r] is already inside the window, jump l just past its previous occurrence.
- Remembering the last index of each character lets l jump in O(1).

# Brute Force

- Check every substring for uniqueness: O(n^3), or O(n^2) with a set per start.

# Optimal Approach

- Last[c] = last index of c (initially -1); l = 0.
- For r: if last[s[r]] >= l, set l = last[s[r]] + 1.
- Last[s[r]] = r; best = max(best, r - l + 1).

# Worked Example

```
s = "abba".

r=0 a  l=0  last a=0  len 1
r=1 b  l=0  last b=1  len 2
r=2 b  seen at 1>=0 -> l=2  len 1
r=3 a  seen at 0 < l=2 -> keep l=2  len 2
best = 2
```

# Java Solution

```java
public class LongestSubstringWithoutRepeating {
    public int lengthOfLongestSubstring(String s) {
        int[] last = new int[128];
        Arrays.fill(last, -1);
        int l = 0, best = 0;
        for (int r = 0; r < s.length(); r++) {
            char c = s.charAt(r);
            if (last[c] >= l) l = last[c] + 1;
            last[c] = r;
            best = Math.max(best, r - l + 1);
        }
        return best;
    }
}
```

# Complexity

- Time: O(n). Space: O(charset), 128 for ASCII.

# Pitfalls

- L must never move backwards ("abba" case) - only jump when the old index is inside the window.